



# The Architectural Fallacy

A Critical Evaluation of SOLID Principles  
in Real-World System Design

---

*Through the Lens of Lakoff's Category Theory, Anscombe's Intention,  
and Wittgenstein's Language Games*

System Design • Not For Interviews

Lecture notes – Areg Melik-Adamyan, Petr Kurapov, Lecture 6-7

# The Central Thesis



*SOLID principles, when applied as universal laws rather than contextual heuristics, systematically produce the very complexity they claim to eliminate. They mistake the map for the territory.*



## Anscombe

Intention without context is meaningless. SRP's "one reason" has no grounding.



## Category Theory

Morphisms show that interface segregation destroys composability of the whole.



## Wittgenstein

Abstractions arise from use, not from top-down principle imposition. DIP inverts reality.

# The Philosophical Framework

Three lenses for evaluating design principles against real system behavior



## Anscombe's Intention

An action's description depends on the context of its intentional structure. There is no "single responsibility" without asking: responsibility under which description? The same module can be described as doing one thing or many, depending on the observer's intentional frame.



## Category Theory and Lakoff's categorization

Objects are defined by their morphisms (relationships), not internal structure. A module's identity is its compositional interface—how it transforms and connects. Segregating interfaces severs morphisms, destroying the categorical coherence of the system.



## Wittgenstein's Games

Meaning arises from use in practice, not from essential definitions. "Open for extension, closed for modification" imports a rigid grammar into domains where the language game demands fluidity. Concepts have family resemblance, not fixed essences.

# S Single Responsibility Principle

*"A class should have only one reason to change."*



## The Anscombe Problem

Anscombe showed that the same physical action admits multiple true descriptions simultaneously. A man pumping water is also:

- Moving his arm up and down
- Operating the pump
- Replenishing the water supply
- **Poisoning the inhabitants**

Which description is the "single" responsibility? SRP has no mechanism to decide.



## In Practice: UserService

A UserService that validates, persists, and notifies. Under SRP, split into:

UserValidator + UserRepository +  
UserNotifier + UserOrchestrator

Result: The orchestrator now has knowledge of all three. You haven't removed the coupling—you've just hidden it behind an indirection layer. The system's immanent complexity is unchanged; the accidental complexity increased.

*Insight: "Reason to change" is observer-relative. Without Anscombe's intentional context, SRP is unfalsifiable.*

# O Open-Closed Principle

*"Open for extension, closed for modification."*



## The Category Theory Objection

In category theory, an object's identity is entirely determined by its morphisms—its compositional relationships with other objects. OCP asks us to fix the object (close for modification) while extending its morphisms (open for extension). But changing the morphisms changes the categorical identity of the object. You cannot be both the same thing and a different thing. OCP pretends the internal structure is separable from the external behavior—category theory proves it is not.



## Wittgenstein's Grammar Violation

OCP imports a grammar of formal logic (fixed axioms, derived theorems) into a domain where the language game is evolutionary. Real systems change because the rules of the game change. When business requirements shift, you don't extend—you rewrite the grammar. Unix survived not by being closed for modification but by having modules whose grammars were independent enough to be rewritten without cascade.

# L Liskov Substitution Principle

*"Subtypes must be substitutable for their base types."*

## Lakoff Destroys Substitutability

Lakoff showed that real categories are prototype-based, not definitional. A robin is "more bird" than a penguin. Categories have graded membership and fuzzy boundaries.

LSP assumes an Aristotelian world where categories have necessary and sufficient conditions. But software entities, like natural categories, exhibit family resemblance. A `PostgresRepository` and an `InMemoryRepository` share enough traits to be useful—but they are not substitutable in any rigorous sense (transaction semantics, latency profiles, failure modes all differ).

## The Classic Failure

### Square $\neq$ Rectangle

Geometrically, every square is a rectangle. But as mutable software objects, they violate LSP because `setWidth()` and `setHeight()` break invariants.

*The principle blames the programmer for a failure that is inherent: the categorical structure of the domain does not preserve morphisms under the subtyping functor.*

# I Interface Segregation Principle

*"No client should depend on methods it does not use."*



## ISP Produces Shallow Modules

Ousterhout's deep module principle states that the best modules hide maximum complexity behind minimal interfaces. ISP systematically inverts this: it fragments rich interfaces into many thin ones, each exposing a narrow slice. The result is a proliferation of shallow abstractions where the interface cost approaches the implementation cost. Consider Unix's file descriptor: `read()`, `write()`, `close()` serve every I/O device. ISP would demand `IReadable`, `IWritable`, `ICloseable`—tripling the conceptual surface for zero complexity reduction.

### Before ISP

- 1 interface, 5 methods
- 1 import per consumer
- 5 total concepts to learn

### After ISP

- 5 interfaces, 1 method each
- 3–4 imports per consumer
- 9+ total concepts to learn

# D Dependency Inversion Principle

*"Depend on abstractions, not concretions."*

## Wittgenstein: Abstractions Emerge from Use

DIP commands us to define abstractions before we have concrete implementations to generalize from. But Wittgenstein showed that meaning (and thus useful abstraction) arises from practice—from the patterns of use in a community. An IRepository interface written before any repository exists encodes no real knowledge. It is a language game played without players. The book's lesson: "Don't design abstractions as if they have a hidden, universal essence. Design them from use cases, interactions, and community expectations."

## Anscombe: The Intention Paradox of DIP

Anscombe's chain of intentions reveals the paradox: the intention behind DIP (decouple high-level policy from low-level detail) requires knowing what counts as "high-level" and "low-level." But this distinction is itself a design decision that presupposes the decomposition DIP is meant to guide. The principle is circular—it tells you to depend on abstractions that you can only define correctly after you have the concrete system DIP says you shouldn't depend on.



# The Complexity Amplification Pattern

How SOLID systematically converts immanent complexity into accidental complexity



## The Parameter Graph Argument

From the book's complexity model: a system is described by parameters  $\lambda_j$  and their relations. Immanent complexity is the minimum graph needed for the system's function  $F(x)$ . SOLID principles, applied dogmatically, do not reduce  $|\{f_k\}|$  (the number of functions). They increase  $|\{\lambda_j\}|$  (parameters) by adding interfaces, abstract classes, factories, and indirection layers. The graph becomes denser, not simpler. Each SRP split adds nodes. Each DIP abstraction adds edges. Each ISP segregation multiplies interface vertices. The entropy of the system description increases even as each local module appears "cleaner."

|              | Files | Interfaces | Dependencies | Cognitive Load |
|--------------|-------|------------|--------------|----------------|
| Before SOLID | 12    | 3          | 18           | Low            |
| After SOLID  | 47    | 19         | 63           | High           |

*Typical mid-size service (User Management domain) refactored to "full SOLID compliance"*

# Systems That Violated SOLID and Won

## Unix

God-interface: fd = open/read/write/close for everything. Violates ISP, SRP.

Deep module: one interface, enormous hidden complexity. The file descriptor's "fat" interface is what enables composition via pipes.

## Git

Blob/tree/commit are concrete types, not abstractions. Violates DIP.

Content-addressable storage is the concept. The concretions ARE the abstraction. The form emerged from the operand (files over time).

## Linux Kernel

Massive modules with mixed concerns. Violates SRP at every level.

Volatility-based decomposition: drivers encapsulate hardware change, scheduler encapsulates policy change. Cohesion by volatility, not by textbook responsibility.

## SQLite

Single-file, monolithic. Violates OCP, ISP, DIP.

2M+ lines in one amalgamation file. The simplest deployment model wins because the system's grammar demands self-containment.



# What Works Instead

Design principles grounded in system reality, not academic formalism



## Volatility-Based Decomposition

Identify what changes and encapsulate it. Don't split by "responsibility" – split by rate of change. This is the book's central thesis: isolate areas subject to frequent change, minimize cascading impact. The decomposition mirrors the system's evolutionary pressure, not an abstract taxonomy.



## Deep Modules over Thin Interfaces

Maximize the ratio of hidden complexity to interface surface area. Unix's `fd`, C's `malloc()`, TCP's socket API. These are "SOLID violations" that have survived 50 years because they are cognitively ergonomic—they match how developers actually work (Lakoff's embodied cognition).



## Concept-Driven Architecture

Map function to form through concepts (refrigerator = box that chills). Let the operand and its transformation guide decomposition. Git's concept is a content-addressable versioned filesystem – the form (blobs, trees, commits) follows naturally. No DIP needed.



## Grammar-Respecting Boundaries

Each subsystem has its own language game with its own grammar. Don't force the scheduler's grammar onto the storage layer (Wittgenstein). Respect that different parts of the system play different games—the boundary is where the grammar changes.

# The Architect as Cartographer, Not Legislator



*Wittgenstein redefined philosophy as therapy: clarifying practices, describing language games, laying out family resemblances. Not inventing hidden essences. The architect's role is not to decree perfect abstractions from on high but to map practices, reveal grammars, and provide clear interfaces.*

Impose principles

→ Map existing practices

Observe how developers actually compose modules, then formalize what works.

Classify by essence

→ Group by family resemblance

A "Service" doesn't need one defining trait. Overlapping similarities suffice.

Legislate structure

→ Reveal natural boundaries

Where the grammar changes, that's where the module boundary belongs.

# Conclusion

SOLID principles are not wrong. They are incomplete. They describe local hygiene rules for code organization, but they are silent on the only question that matters:

How do you decompose a system so that its structure reflects its evolutionary pressures?

**Intention grounds responsibility.** Without Anscombe's context, SRP is vacuous.

**Morphisms define identity.** Category theory shows ISP/OCP fragment coherence.

**Meaning is use.** Wittgenstein shows DIP inverts how abstractions actually form.

*Design is not a checklist. It is a practice.*



# Beyond Conventional Wisdom

Volatility-Based Design and Alternative  
Decomposition Strategies for Real Systems

---

*Why the systems that survived did not follow the textbook—  
and what they did instead*

# The Conventional Wisdom Trap

Three decomposition strategies that every textbook teaches—and why each one breaks under real evolutionary pressure



## Functional Decomposition

Mirrors requirements, not change patterns. When requirements shift, the entire decomposition cascades. A house decomposed by function (cooking, sleeping, bathing) must be restructured when you add a grill—because 'cooking' now spans two physical spaces.

*Component explosion, orchestrator bloat, tight coupling between "independent" modules*



## Domain Decomposition (DDD)

Aligns with business language, but business domains are not stability boundaries. When the Kitchen domain needs a feature from the Living Room domain, you either duplicate logic or create cross-domain coupling that the bounded context was meant to prevent.

*Reimplemented functionality, rigid domain boundaries that resist real-world overlap*



## Object-Oriented Decomposition

Not all problems are objects. Graph algorithms, data pipelines, and protocol handlers resist class hierarchies. Excessive OO decomposition produces deep inheritance trees where changes at the root ripple to every leaf.

*Complex inheritance hierarchies, performance overhead, alien abstractions*

# Why "By Function" Has No Ground

Anscombe's Intention and the observer-relativity of decomposition



## The Chain of Why-Questions

Anscombe (1957) showed that an action's identity depends on which description you choose. The same code module can be described as:

- "Handling user authentication" (function)
- "Isolating the OAuth volatility" (volatility)
- "Implementing the User aggregate" (domain)
- "Encapsulating the auth API" (information hiding)

Each description is simultaneously true. Functional decomposition picks one and pretends the others don't exist. The question isn't what does this module do but under which intention is it organized?

## The Practical Cost

A payment system decomposed by function:

```
ValidatePayment  
ProcessPayment  
NotifyPayment  
LogPayment
```

When the payment provider changes, all four modules change. The functional boundary is orthogonal to the actual axis of change.

*Insight: The right decomposition is the one whose boundaries align with the system's axes of volatility, not its axes of function.*



# Volatility-Based Decomposition

Organize your system around what changes, not what it does

“ *The core idea: identify the areas of your system that are subject to frequent change—data formats, user interfaces, business rules, external APIs, hardware targets—and encapsulate each volatile area in a separate module. The module boundary becomes a firewall: changes inside the module do not propagate outside it.* ”

This is Ashby's Law of Requisite Variety applied to software: the regulator (your architecture) must have at least as much variety as the disturbance (change) it needs to absorb. Volatility-based boundaries give you that variety exactly where you need it.

1



## Identify Axes of Volatility

During requirements analysis, ask: what will change? User needs, client expectations, operating environments, content types, regulatory requirements—each is a potential volatility axis.

2



## Encapsulate Behind Stable Interfaces

Each volatile area gets a module whose interface is stable even as its internals evolve. The interface is the contract; the implementation is the variable.

3



## Compose via Deep Modules

The resulting modules should be deep (Ousterhout): maximum hidden complexity, minimal interface surface. The composition of deep modules produces systems that are both flexible and comprehensible.



# Case Study: Git's Decomposition

How one system combines three strategies into a coherent architecture

## Concept: A content-addressable, versioned filesystem

Operand: Files and directories • Attribute: Content over time • Function: Track and retrieve historical versions

Form: Commits, blobs, and trees connected via content-addressed hashes. The concept drove the form—no DIP, no ISP, no abstract factory.

### Functional

commit, branch, merge, rebase, diff, log—each a distinct command with its own entry point.

*Provides the user-facing grammar. Each command is a verb in Git's language game.*

### Object-Oriented

blob, tree, commit, tag—concrete types with specific responsibilities. No abstract base class.

*Provides the data model. The objects ARE the abstractions. Content-addressing eliminates the need for interfaces.*

### Volatility-Based

Transport (HTTP/SSH/local), storage (packfiles/loose objects), UI (CLI/GUI), merge strategy.

*Provides the resilience. Each volatile area is encapsulated: changing transport doesn't touch storage.*

*Git doesn't pick one strategy. It layers them: functional for the CLI grammar, OO for the data model, volatility for the architecture.*

# Case Study: Unix's File Descriptor

The "god interface" that has survived 55 years

## The Interface

`open()` `read()` `write()` `close()`

Four functions serve:

- Regular files
- Directories
- Block devices
- Character devices
- Network sockets
- Pipes
- /proc virtual filesystem

## Why It Works

Ousterhout's depth ratio: 4 interface functions hiding thousands of lines of kernel implementation per device type. The interface cost is vanishingly small relative to the hidden complexity.

Lakoff's cognitive fit: developers already have the metaphor of "reading" and "writing" from literacy. The abstraction maps to embodied human experience.

Wittgenstein's grammar: every I/O device plays the same language game. Pipes compose because they share the fd grammar—`cat` | `sort` | `uniq` works because every participant speaks the same rules.

*ISP would demand `IReadable`, `IWritable`, `ISeekable`, `ICloseable`. The result: 4× the interface cost, zero complexity reduction, and no composition via pipes.*

# Case Study: The Linux Kernel

30M+ lines, 30+ years of evolution – decomposed by volatility, not by textbook

**drivers/**     Hardware changes → Driver model API (register, probe, remove)

Hardware is the most volatile axis. 60%+ of the kernel is drivers, each encapsulating a specific hardware target behind a stable interface.

**fs/**             Filesystem formats → VFS interface (inode, dentry, superblock)

ext4, btrfs, NFS all change independently behind the VFS. New filesystems don't touch the scheduler.

**net/**             Protocol evolution → Socket API

TCP, UDP, QUIC—protocols evolve without touching the user-space interface.

**arch/**             CPU architectures → Kernel internal API

ARM, x86, RISC-V each in their own directory. Architecture-specific code is firewalled from generic kernel logic.

*The kernel's grammar: each subsystem plays its own language game. The VFS doesn't impose filesystem semantics; it provides a shared interface that each filesystem implements in its own grammar.*



# Case Study: SQLite

The most-deployed database in the world is a single-file monolith

## What SQLite Violates

- SRP: SQL parser + B-tree + pager + OS layer in one codebase
- OCP: Changes modify existing code freely
- ISP: Single monolithic API
- DIP: Concrete types throughout, no abstract interfaces
- Conventional modularization: distributed as a single amalgamation file (sqlite3.c)

## Why It Wins

The system's grammar demands self-containment. SQLite's operand is "a relational database in a single file." The concept requires zero external dependencies, zero server processes, zero configuration.

The volatility is internal: the B-tree implementation, page cache strategy, and query optimizer can all change without affecting the single-file deployment model. The architecture serves the concept, not a principle.

**1T+**

Active deployments

**150K+**

Lines of code

**100%**

Branch coverage

**0**

External dependencies

*Wittgenstein's lesson: the abstraction's meaning depends on its use, not its internal structure. SQLite's users need a single-file embeddable DB. The monolith **\*IS\*** the abstraction.*



# Alternative 1: Concept-Driven Architecture

From the book's Chapter 4: Map function to form through concepts

## The Operand → Function → Concept → Form Pipeline

Instead of starting with principles (SOLID) or patterns (GoF), start with the entity your system transforms (the operand) and the attribute you need to change. The concept is the bridge between what and how: it names the essential idea that links the required transformation to a possible structure. Once you have the concept, the form, the actual modules, data structures, interfaces emerges naturally.

### Refrigerator

|            |                                            |
|------------|--------------------------------------------|
| Operand:   | Food                                       |
| Attribute: | Spoilage rate                              |
| Function:  | Slow spoilage                              |
| Concept:   | Insulated box maintaining cold environment |
| Form:      | Compressor + insulation + thermostat       |

### Git

|            |                                          |
|------------|------------------------------------------|
| Operand:   | Files & directories                      |
| Attribute: | Content over time                        |
| Function:  | Track & retrieve versions                |
| Concept:   | Content-addressable versioned filesystem |
| Form:      | Blobs, trees, commits via SHA hashes     |

### TCP

|            |                                                  |
|------------|--------------------------------------------------|
| Operand:   | Byte stream                                      |
| Attribute: | Reliable delivery                                |
| Function:  | Guarantee ordered arrival                        |
| Concept:   | Virtual reliable channel over unreliable network |
| Form:      | Segments, ACKs, windows, retransmission          |



# Alternative 2: Grammar-Respecting Boundaries

From Wittgenstein: each subsystem has its own language game with its own rules

The key insight: confusion arises when you import rules from one game into another. The scheduler has different rules than the compiler IR; the storage system has different rules than the networking stack. A module boundary should be placed where the grammar changes, where the rules of one language game end and another's begin. Misaligned decomposition comes from mixing grammars, like treating logging as synchronous state mutation when its grammar is asynchronous observation.

## Database Engine

Transactions, ACID, isolation levels

X Imposing database consistency semantics on a real-time streaming pipeline

## Network Protocol

Packets, acknowledgments, timeouts

X Treating HTTP requests like synchronous function calls in a distributed system

## Compiler

Parse trees, IRs, optimization passes

X Using OOP class hierarchies for an inherently tree-transforming pipeline

## Build System

Targets, dependencies, recipes, DAG

X Modeling build rules as objects with inheritance instead of graph nodes

# Alternative 3: Cognitive Fit

Lakoff's embodied cognition: abstractions must feel natural to their users

Categories are prototype-based, not definitional. A robin is "more bird" than a penguin. When you design an abstraction, it must have a clear prototype: a central, typical use case that anchors the concept. Peripheral cases radiate outward as extensions, not as inheritance branches.

If an abstraction forces developers to think in alien terms, it will resist adoption regardless of its theoretical elegance. PyTorch's Tensor works because it matches both the mathematical prototype (n-d array) and the programming metaphor. SOLID's abstract factories don't work because nobody thinks in factory-of-factory terms.

## Tensor (PyTorch)

Matches math prototype + programming metaphor

## AbstractSingletonProxyFactoryBean

No human prototype for this concept

## File descriptor (Unix)

read/write maps to literacy experience

## IReadableStreamProvider

Cognitive overhead exceeds value

## DataFrame (Pandas)

Spreadsheet metaphor everyone knows

## DataAccessObjectFactory

Concept defined by architecture, not use



# Decomposition Strategies Compared

| Strategy         | Organizes By         | Handles Change        | Best For               | Fails When               |
|------------------|----------------------|-----------------------|------------------------|--------------------------|
| Functional       | What system does     | Poorly-cascading      | Requirements discovery | Requirements change      |
| Domain (DDD)     | Business language    | Cross-domain pain     | Business alignment     | Domains overlap          |
| Object-Oriented  | Entity encapsulation | Deep hierarchy ripple | Data modeling          | Non-OO problems          |
| Volatility-Based | What changes         | Localized by design   | Long-lived systems     | Volatility misidentified |
| Concept-Driven   | Operand → form       | Form follows concept  | Greenfield design      | Unclear operand          |
| Grammar-Based    | Language game rules  | Grammar isolation     | Multi-paradigm systems | Single-paradigm          |

*Green rows: approaches grounded in system reality. The right strategy depends on context, but volatility analysis is the universal first question.*



# The Architect's Decision Framework

## 1. What is the operand, and what transformation does the system perform?

→ Concept-Driven Architecture

This gives you the concept, the bridge from function to form. Without it, you're decomposing a system you haven't understood.

## 2. What changes, and at what rate?

→ Volatility Analysis

Map every known source of change: hardware, protocols, business rules, UI, regulations. Each axis of volatility becomes a module boundary candidate.

## 3. Which subsystems speak different languages?

→ Grammar Analysis

Where the rules change, the boundary belongs. The scheduler's grammar is not the storage layer's grammar. Forcing one onto the other is Wittgenstein's 'language going on holiday.'

## 4. Does the abstraction match how developers think?

→ Cognitive Fit Test

If the abstraction requires a tutorial to explain, it has failed Lakoff's test. Good abstractions anchor in prototypes developers already know.

# Conclusion

Conventional decomposition strategies organize systems by what they are or what they do.

The systems that survived organized by what changes.

Start with the concept: operand → function → concept → form.

Decompose by volatility: boundaries where change is, not where function is.

Respect each subsystem's grammar: don't import rules from one game into another.

Test for cognitive fit: if the abstraction needs a manual, redesign it.

*Design is not a checklist. It is a practice grounded in understanding of the system, of change, and of the people who build it.*